



Pizza Sales Analytics — SQL Project

Pizzerias accumulate vast amounts of order data daily, but extracting meaningful insights about sales performance and customer preferences can be challenging.

This SQL project delves into a comprehensive pizza sales dataset, utilizing advanced queries to uncover trends, optimize menu offerings, and understand customer purchasing habits.

Project Goals

Validate data quality

Identify NULLs, inconsistent prices, and missing fields that affect revenue calculations.

Compute key metrics

Orders, revenue, quantities, time-of-day patterns, and top-selling pizzas.

Deliver actionable insights

Operational recommendations for menu, pricing, and staffing based on results.





Database

Data Sources & Schema

Primary tables:

- orders (order_id, order_date, order_time),
- order_details (order_id, pizza_id, quantity),
- pizzas (pizza_id, pizza_type_id, size, price),
- pizza_types (pizza_type_id, name, category, ingredients).

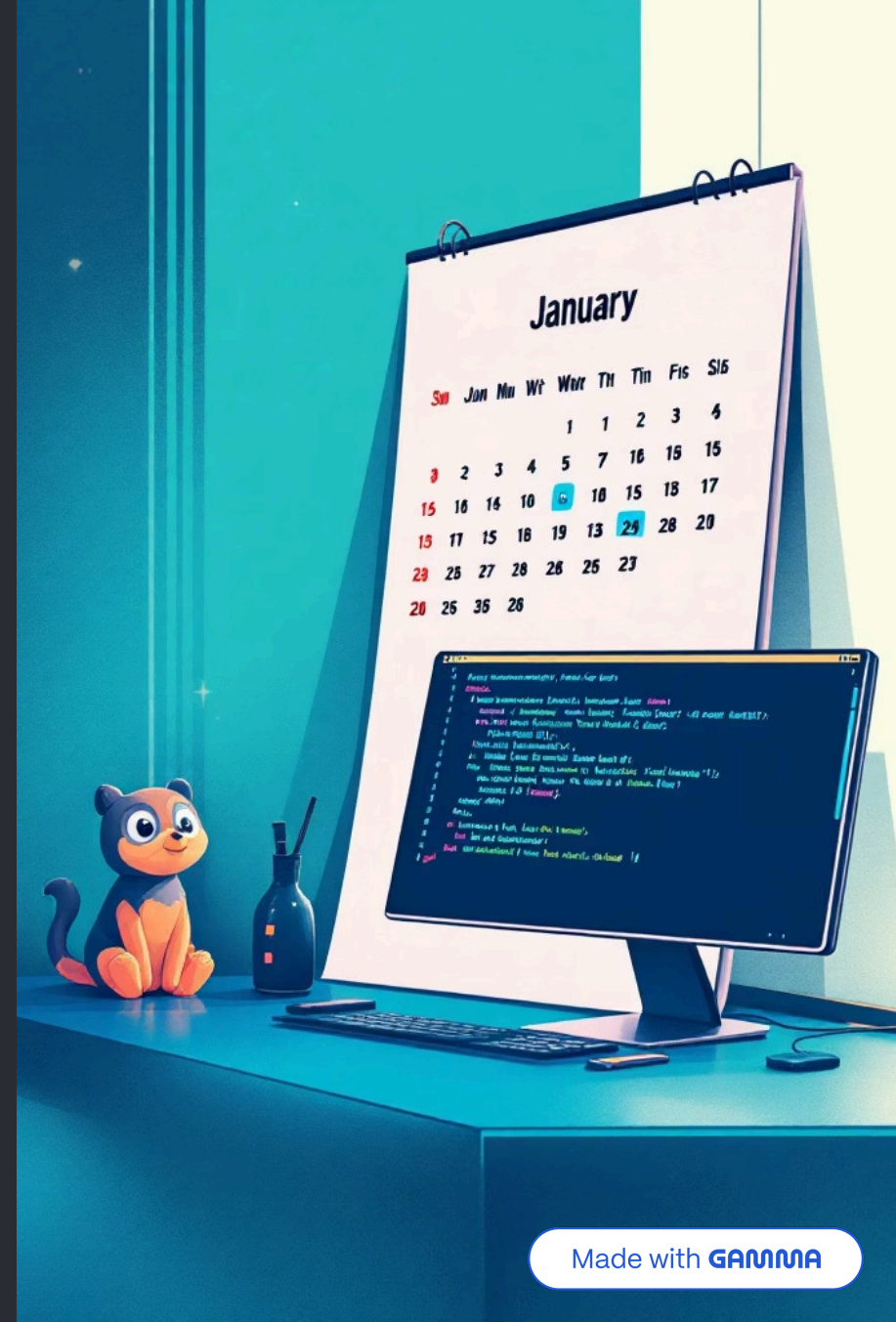
Joins are standard PK→FK.

Sample Query: Monthly Order Count

Goal: Count total orders in January 2015.

```
SELECT COUNT(order_id) FROM orders WHERE order_date BETWEEN  
'2015-01-01' AND '2015-01-31';
```

❏ Use **DISTINCT** order_id if orders can appear multiple times in the table.



Filtering by Pizza Size — Example



Goal: List orders where a large (L) pizza was sold.

```
SELECT pt.name, o.order_id, o.order_date, o.order_time FROM orders o JOIN  
order_details od ON od.order_id = o.order_id JOIN pizzas p ON od.pizza_id =  
p.pizza_id JOIN pizza_types pt ON pt.pizza_type_id = p.pizza_type_id WHERE p.size  
= 'L';
```

Include DISTINCT on (o.order_id, pt.name) to avoid duplicate rows when multiple large pizzas exist per order.

Revenue Calculations



Revenue per Order

```
SELECT o.order_id, ROUND(SUM(od.quantity * p.price), 2) AS revenue
FROM orders o JOIN order_details od ON o.order_id = od.order_id JOIN
pizzas p ON od.pizza_id = p.pizza_id GROUP BY o.order_id;
```



Daily Revenue (Top Days)

```
SELECT ROUND(SUM(quantity * price), 2) AS revenue, o.order_date FROM
orders o JOIN order_details od ON o.order_id = od.order_id JOIN pizzas p
ON od.pizza_id = p.pizza_id GROUP BY o.order_date ORDER BY revenue
DESC;
```



Sales Volume & Top Items



Top-selling pizza by quantity:

```
SELECT p.pizza_id, SUM(quantity) AS  
total_quantity FROM pizzas p JOIN  
order_details od ON p.pizza_id =  
od.pizza_id GROUP BY p.pizza_id ORDER  
BY SUM(quantity) DESC LIMIT 1;
```

Rank all pizzas by quantity:

```
SELECT pizza_id, SUM(quantity) AS  
total_quantity, RANK() OVER (ORDER BY  
SUM(quantity) DESC) AS quantity_rank  
FROM order_details GROUP BY pizza_id;
```

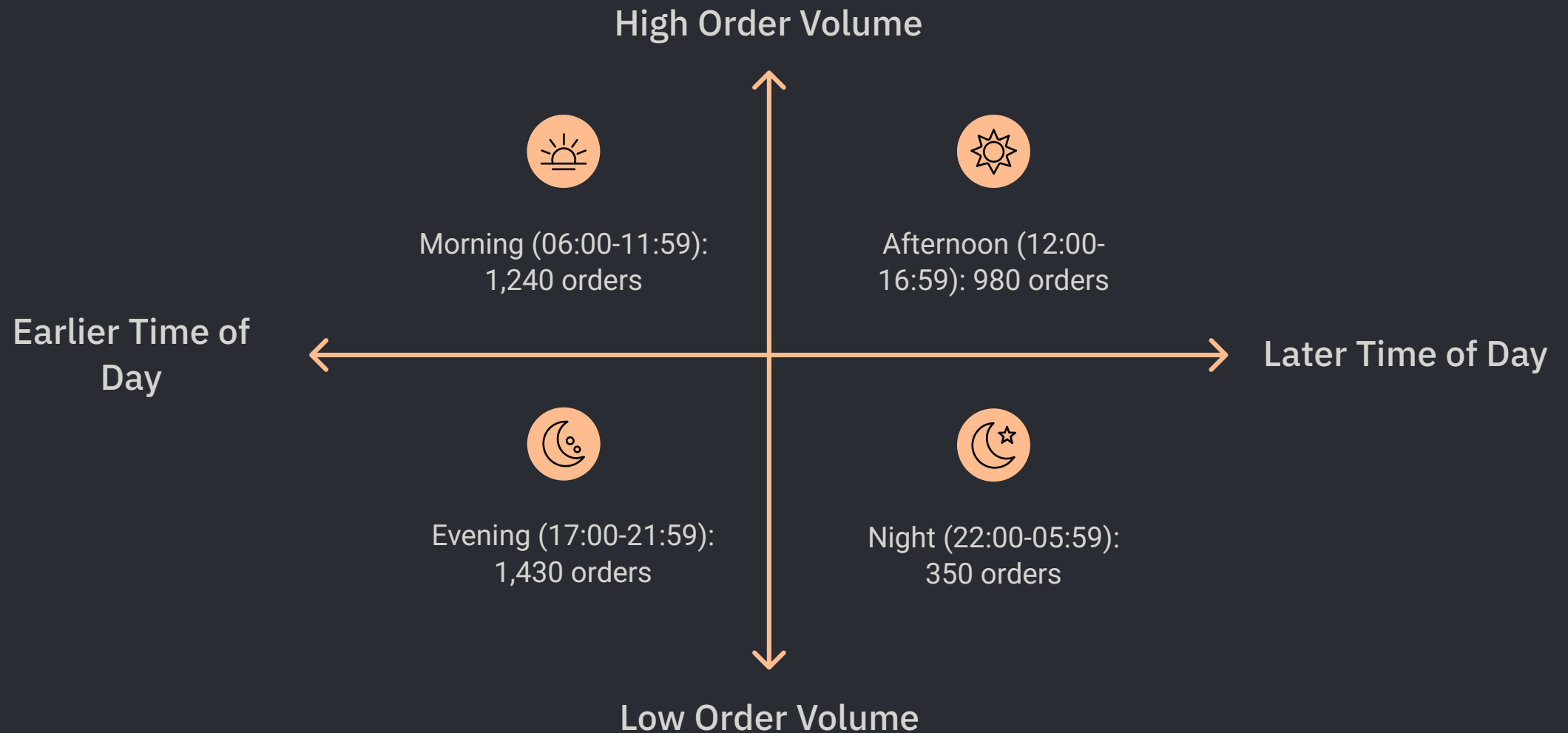



Category Performance

```
SELECT pt.category, SUM(od.quantity) AS total_quantity,  
ROUND(SUM(p.price * od.quantity), 2) AS total_revenue FROM orders o  
JOIN order_details od ON o.order_id = od.order_id JOIN pizzas p ON  
od.pizza_id = p.pizza_id JOIN pizza_types pt ON p.pizza_type_id =  
pt.pizza_type_id GROUP BY pt.category;
```

Use this to prioritize menu categories for promotions or cost analysis.

Time-of-Day Analysis



SQL pattern:

```
SELECT COUNT(order_id) AS cnt, CASE WHEN order_time BETWEEN '06:00:00' AND '11:59:00' THEN 'Morning' WHEN order_time BETWEEN '12:00:00' AND '16:59:00' THEN 'Afternoon' WHEN order_time BETWEEN '17:00:00' AND '21:59:00' THEN 'Evening' ELSE 'Night' END AS time_of_day FROM orders GROUP BY time_of_day;
```

Data Quality Checks & Advanced Windowing



- Find NULL prices

```
SELECT od.order_id, p.price FROM  
order_details od JOIN pizzas p ON  
od.pizza_id = p.pizza_id WHERE  
p.price IS NULL;
```

- Running totals & window functions

```
SELECT order_id, SUM(quantity) OVER  
(ORDER BY order_id ROWS BETWEEN  
UNBOUNDED PRECEDING AND  
CURRENT ROW) AS running_qty FROM  
order_details;
```

- Price rank within category

```
SELECT p.pizza_id, pt.category,  
p.price, RANK() OVER (PARTITION BY  
pt.category ORDER BY p.price DESC)  
AS rank_in_category FROM pizzas p  
JOIN pizza_types pt ON  
p.pizza_type_id = pt.pizza_type_id;
```

- ❏ Resolve NULL prices before aggregations to avoid skewed revenue; use COALESCE or exclude as appropriate.